

Build Troubleshooting Guide

Common errors encountered during AllianceMine builds and how to fix them.

Memory / OOM Errors

Symptom

Gradle build daemon disappeared unexpectedly or Java OOM during FASTA integration (especially alliance-xlaevis-fasta, fbbt).

Cause

- The alliancemine repo hardcodes `org.gradle.jvmargs=-Xmx48g` in `gradle.properties`, overriding any `GRADLE_OPTS`.
- Docker container memory limit must exceed JVM heap by ~8 GB (for metaspace, native memory, stacks).

Fix

The Dockerfile already patches this. Make sure `docker-compose.yml` has:

```
GRADLE_OPTS: -Xmx48g -Xms24g ...
deploy:
  resources:
    limits:
      memory: 56G
```

project_build Failures

Symptom

Build fails during `project_build` with cryptic errors, or resume doesn't work.

Flag cheat sheet

Flag	Use
-b	Run <code>builddb</code> before integration. DROPS all backup databases. Use only for fresh builds.
-l	Load last checkpoint and resume. Never combine with -b.
-r	Restart without loading (re-integrates from source). Wrong for most resumes.
-T	Never use. Disables server-side CREATE DATABASE TEMPLATE backups, forces slow <code>pg_dump</code> files.
-E UTF8	Required for RDS. Default <code>SQL_ASCII</code> breaks <code>createdb</code> with locale errors.

Correct invocations

- **Fresh build:** `./project_build -b -E UTF8 localhost /root/data/dump`
- **Resume after failure:** `./project_build -l -E UTF8 localhost /root/data/dump`

SGD Source Failures

Symptom: Duplicate objects found for pk ... TelomericRepeat

Cause: `SgdConverter.java` has commented-out dedup logic for child features. The SGD SQL query returns duplicate `telomeric_repeat` rows.

Fix: The Dockerfile patches this automatically. Upstream fix is needed in `alliance-genome/alliancemine-bio-sources`.

Symptom: Connection refused to localhost:5432 on sgd source

Cause: `SGD_DB_*` env vars not passed through to the container.

Fix: Ensure `docker-compose.yml` has the SGD env vars in the `environment:` block AND they're set in `.env`.

Symptom: sgd-protein-properties fails with duplicate Protein items with secondaryIdentifier like 3324.9

Cause: `Old extract_data.py` was downloading `protein_properties.tab` from `downloads.yeastgenome.org`. This file has **40 columns**, but the converter expects 7. Column 2 is Mw (molecular weight) which became `secondaryIdentifier`.

Fix: Don't download it. The current `extract_data.py` only does S3 sync which uses `wal-lace2015_heat_aggregation_annotations_Kyla.txt`.

Solr Failures

Symptom: create-search-index connects to localhost:8983/solr/alliancemine-search regardless of properties

Cause: `keyword_search.properties` in the `alliancemine` repo hardcodes `index.solrurl`.

Fix: Patch it via `sed` inside the running container:

```
docker exec <container> sed -i \  
  "s|index.solrurl = http://localhost:8983/solr/alliancemine-search|index.solrurl = \  
  ↪ http://172.31.59.87:8983/solr/alliancemine-search-9.0.0|" \  
  /root/alliancemine/dbmodel/resources/keyword_search.properties
```

Symptom: create-autocomplete-index indexes SOTerm, DOTerm, GOTerm then fails

Cause: `objectstoresummary.config.properties` hardcodes `autocomplete.solrurl`. The indexing starts working (reads field list from this file) then hits the hardcoded URL.

Fix: Same as above but for `autocomplete`:

```
docker exec <container> sed -i \
"s|autocomplete.solrurl =
↪ http://localhost:8983/solr/alliancemine-autocomplete|autocomplete.solrurl =
↪ http://172.31.59.87:8983/solr/alliancemine-autocomplete-9.0.0|" \
/root/alliancemine/dbmodel/resources/objectstoresummary.config.properties
```

Symptom: Solr cores don't exist before postprocess

Cause: InterMine does NOT auto-create Solr cores. The build's preflight step normally creates them (build_full.py calls create_solr_cores.py before stage 1), but the preflight is warn-only — if it failed, the build continued and postprocess is now hitting the missing cores.

Fix: Re-run the preflight manually, then resume:

```
# Inside the build container shell:
python3 /root/scripts/create_solr_cores.py \
  --release "${ALLIANCE_RELEASE}" \
  --rc "${RC_NUMBER}" \
  --solr-host "${SOLR_HOST}"
# Add --ssh-key /root/.ssh/AGR-ssl3.pem if running from outside AWS
```

Then restart the build with `--start-from postprocess --resume`.

Symptom: Solr preflight warns “Permission denied (publickey)” or “Connection refused”

Cause: The build container can't SSH to the multitenant host. Inside AWS this is usually a security-group / VPC issue; from outside AWS it's a missing or wrong SOLR_SSH_KEY.

Fix: - **Inside AWS:** confirm the multitenant host's `~ec2-user/.ssh/authorized_keys` contains the AllianceMineDev key. The compose file mounts `~/ .ssh` read-only; check it actually contains usable keys (docker compose run --rm alliancemine-builder bash -c 'ls /root/.ssh'). - **From a laptop:** set `SOLR_SSH_KEY=/root/.ssh/AGR-ssl3.pem` in `.env` and confirm `~/ .ssh/AGR-ssl3.pem` is `chmod 600` on the host.

The build will continue regardless — the preflight is non-fatal. Postprocess will fail later with “core not found” if you don't fix it before then.

Webapp Errors

Symptom: HTTP 503 Cannot set superuser.account 'admin@alliancemine.org' to be superuser. Does this profile exist?

Cause: Profile DB (db.userprofile-production.datasource.databaseName) points to a DB where admin@alliancemine.org doesn't exist. The production profile DB has superuser@mail_account.

Fix: Update the deployed webapp's intermine.properties:

```
docker exec <container> sed -i \
's|superuser.account=admin@alliancemine.org|superuser.account=superuser@mail_account|'
↪ \
/usr/local/tomcat/webapps/alliancemine/WEB-INF/classes/intermine.properties
```

And point profile DB to production:

```
docker exec <container> sed -i \
's|databaseName=alliancemine_userprofile_test|databaseName=alliancemine_userprofile|' \
/usr/local/tomcat/webapps/alliancemine/WEB-INF/classes/intermine.properties
```

Symptom: XML validation failed. <http://localhost:8090/alliancemine/webservice/query.xsd>

Cause: webapp.baseurl in web.properties (inside WEB-INF) points to localhost:8090. The webapp tries to fetch its own XSD from this URL to validate incoming queries.

Fix: Update BOTH properties files (the WEB-INF web.properties overrides intermine.properties):

```
docker exec <container> sed -i \
's|webapp.baseurl=http://localhost:8090|webapp.baseurl=http://localhost:8080|' \
/usr/local/tomcat/webapps/alliancemine/WEB-INF/classes/intermine.properties
```

```
docker exec <container> sed -i \
's|webapp.baseurl=http://localhost:8090|webapp.baseurl=http://localhost:8080|' \
/usr/local/tomcat/webapps/alliancemine/WEB-INF/web.properties
```

Then stop and start the webapp (reload doesn't reread properties reliably):

```
docker exec <container> curl -s -u manager:manager
↪ "http://localhost:8080/manager/text/stop?path=/alliancemine"
docker exec <container> curl -s -m 180 -u manager:manager
↪ "http://localhost:8080/manager/text/start?path=/alliancemine"
```

Symptom: Pages extremely slow after deploy, even simple ones

Cause: Bag upgrade thread is running. On first start with a new database, InterMine upgrades all user-saved gene lists from old IDs to new ones. This thread competes for resources with page requests.

Fix: Wait. For ~100 lists, this takes 10-30 minutes. No way to skip without losing user data.

RDS Storage

Symptom: could not extend file ... No space left on device

Cause: Checkpoint databases (server-side backups via CREATE DATABASE TEMPLATE) fill RDS. Each checkpoint is a full DB copy (~30-60 GB).

Fix: Drop old checkpoints during the build:

```
psql -h ... -d postgres -c 'DROP DATABASE "alliancemine_X_X_X_rcN:checkpoint-name"'
```

Monitor RDS storage:

```
aws cloudwatch get-metric-statistics \
--namespace AWS/RDS --metric-name FreeStorageSpace \
--dimensions Name=DBInstanceIdentifier,Value=intermine-postgres \
--start-time $(date -u -d '5 minutes ago' +%Y-%m-%dT%H:%M:%S) \
--end-time $(date -u +%Y-%m-%dT%H:%M:%S) \
--period 300 --statistics Average \
--query 'Datapoints[0].Average' --output text
```

Connection Timeouts

Symptom: Connection to ... timed out during integration

Cause: Transient network issue with RDS or NCBI (for update-publications).

Fix: Usually recovers on its own. If update-publications fails with Premature EOF, resume from the last checkpoint and try again.

Solr Hostname Resolution in Production

Symptom

java.lang.NullPointerException at SolrKeywordSearchHandler.performSearch on the production AllianceMine homepage (layout.jsp).

Cause

The keyword_search.properties inside dbmodel.jar (baked into the WAR) has:

```
index.solrurl=http://agr.stage.alliancemine.solr.server:8983/solr/alliancemine-search
```

This hostname was a Docker container name from the old setup. Solr now runs on the **host** directly, not in a container. The Tomcat container can't resolve this hostname.

Quick Fix (survives until container restart)

```
docker exec alliancemine sh -c \  
'echo "172.17.0.1 agr.stage.alliancemine.solr.server" >> /etc/hosts'
```

172.17.0.1 is the Docker bridge gateway — routes to the host where Solr runs on port 8983.

Permanent Fix

Recreate the container with the host mapping:

```
docker run -d --name alliancemine \  
  --add-host agr.stage.alliancemine.solr.server:host-gateway \  
  -p 8080:8080 \  
  intermine-tomcat:latest
```

Or fix the keyword_search.properties in the alliancemine repo to not hardcode the hostname (upstream PR pending on fix/build-config-hardcoded-values branch).

Compile Caching

Problem: Every fresh container recompiles for 10-15 min

Fix: After the first successful compilation, commit the container state as a new image:

```
docker commit <container_name> alliancemine-builder:9.0.0-compiled
```

Update `docker-compose.yml` to use the compiled image:

```
image: alliancemine-builder:9.0.0-compiled
```

For script-only changes, patch the compiled image in place:

```
docker create --name tmp alliancemine-builder:9.0.0-compiled bash
docker cp /path/to/build_full.py tmp:/root/scripts/build_full.py
docker commit tmp alliancemine-builder:9.0.0-compiled
docker rm tmp
```